

ADR 0001 – Adopter Drizzle ORM pour la couche de persistance

- **Statut** : Proposé
- **Date** : 2026-09-17
- **Décideurs** : à compléter
- **Remplace** : la recommandation « Prisma, TypeORM ou Sequelize » de l'audit, section 7.2
- **Documents liés** : Audit `translate.md` (sections 2.2, 7.2, 7.3, 7.8), `src/scripts/mergeSqliteIntoMysql/`

Contexte

L'application est en production. Sa couche de persistance présente les problèmes relevés par l'audit :

1. **Requêtes dupliquées.** `src/persistence/sqlite.ts` et `src/persistence/mysql.ts` réimplémentent les mêmes opérations (`getItems`, `getItem`, `storeItem`, `updateItem`, `removeItem`). Le moteur est choisi selon la présence de `MYSQL_HOST`.
2. **Pas de migrations.** Les tables sont créées au démarrage par des `CREATE TABLE IF NOT EXISTS` écrits à la main. `todo_items` n'a ni clé primaire ni contrainte `NOT NULL`.
3. **Deux accès MySQL parallèles.** Le chemin événementiel (`src/infrastructure/db/mysql.ts`) a son propre pool `mysql2/promise`, parce que l'adaptateur historique n'offre pas de transactions.
4. **Un outbox transactionnel déjà en place.** `updateTask` écrit la tâche et l'événement dans la même transaction et verrouille la ligne avec `SELECT ... FOR UPDATE`. Le relay réclame les événements avec `FOR UPDATE SKIP LOCKED` pour

pouvoir tourner en plusieurs instances.

5. **La cible est MySQL 8.4 uniquement** (`docker-compose.yml`). SQLite doit disparaître, avec son module natif `sqlite3` qui oblige à installer python, make et g++ dans l'image.
6. **La stack** : TypeScript strict, Node 24, Express 5, Zod, image Docker multi-étapes compilée avec `tsc` .

L'audit recommande un ORM avec migrations versionnées, sans avoir tranché : il cite Prisma, TypeORM et Sequelize.

Critères de décision

- **C1 – Verrous et transactions.** Écrire `FOR UPDATE` et `FOR UPDATE SKIP LOCKED` dans des transactions interactives sans passer par du SQL brut, parce que l'outbox en dépend.
- **C2 – Typage TypeScript** des résultats de requêtes, sans cast manuel.
- **C3 – Migrations versionnées**, relisibles en SQL et compatibles avec une base existante (point de départ sans recréer les tables).
- **C4 – Migration progressive** d'une app en production : réutiliser `mysql2` , cohabiter avec le code actuel, basculer module par module.
- **C5 – Impact sur le build et l'image Docker** : pas de binaire natif ni d'étape de génération de code en plus.
- **C6 – Contrôle du SQL généré**, parce que le schéma a des besoins précis (`DATETIME(3)` , colonnes `JSON` , index composites, `utf8mb4`).
- **C7 – Validation** : pouvoir dériver des schémas Zod à partir du modèle.

Options étudiées

Option A – Drizzle ORM (retenue)

Schéma déclaré en TypeScript, API proche du SQL, migrations avec `drizzle-kit`.

- **C1** : le query builder MySQL gère directement `.for('update')` et `.for('update', { skipLocked: true })`. Les transactions se font avec `db.transaction(async tx => ...)`.
- **C2** : les types sont déduits du schéma, sans génération de code.
- **C3** : `drizzle-kit generate` produit des fichiers SQL relisibles. `drizzle-kit pull` lit une base existante pour créer le schéma de départ.
- **C4** : fonctionne au-dessus de `mysql2`, déjà utilisé. `withTransaction`, `enqueue`, `claimBatch` et `updateTask` se transposent presque ligne à ligne.
- **C5** : bibliothèque TypeScript/JavaScript seulement, sans binaire natif ni `generate`.
- **C6** : le SQL émis correspond à ce qui est écrit.
- **C7** : des schémas Zod peuvent être dérivés des tables.
- **Points faibles** : le schéma dépend du moteur (`mysqlTable` $\neq$ `sqliteTable`). L'API relationnelle de haut niveau est moins riche que celle de Prisma. L'outil est plus jeune et évolue encore vite : il faut figer les versions.

Option B — Prisma

Fichier `schema.prisma` et client généré.

- **Points forts** : excellente expérience de développement, schéma très lisible, migrations robustes, grande communauté. Changer de moteur se fait surtout par la configuration. C'est l'option que le code actuel mentionne en commentaire.
- **C1** : l'API ne gère pas `FOR UPDATE / SKIP LOCKED`. Le cœur de l'outbox passerait par `$queryRaw` dans `$transaction`, et on perdrait le typage justement sur le code le plus sensible.
- **C5** : ajoute une étape `prisma generate` au build, à la CI et au Dockerfile.
- **C6** : plus de distance entre le code et le SQL exécuté.

- **Conclusion** : bonne option générale, moins adaptée à un projet dont le composant critique repose sur des verrous précis.

Option C — TypeORM

- Typage moins fiable (décorateurs, `reflect-metadata`), migrations générées souvent à corriger, maintenance jugée moins active.
- N'apporte rien de plus que A ou B sur nos critères.

Option D — Sequelize

- Conçu à l'origine pour JavaScript, typage TypeScript approximatif, API éloignée du SQL.
- Écarté.

Option E — Kysely (query builder)

- Très bon typage, proche du SQL, gère les verrous.
- Pas de déclaration de schéma ni d'outil de migration complet intégré : ne répond qu'en partie au besoin de l'audit (C3).

Option F — Garder mysql2 à la main

- Aucun coût de migration, mais ne règle ni la duplication ni l'absence de migrations. Contraire aux recommandations de l'audit.

Synthèse

Critère	Drizzle	Prisma	TypeORM	Sequelize	Kysely	Statu quo
---------	---------	--------	---------	-----------	--------	-----------

Critère	Drizzle	Prisma	TypeORM	Sequelize	Kysely	Statu quo
C1 Verrous / transactions	Oui	Partiel (SQL brut)	Partiel	Partiel	Oui	Oui (non typé)
C2 Typage	Oui	Oui	Partiel	Non	Oui	Non
C3 Migrations	Oui	Oui	Partiel	Partiel	Partiel	Non
C4 Migration progressive	Oui	Partiel	Partiel	Partiel	Oui	—
C5 Build / image	Oui	Partiel (étape generate)	Oui	Oui	Oui	Oui
C6 Contrôle du SQL	Oui	Partiel	Partiel	Non	Oui	Oui
C7 Zod	Oui	Partiel (outil tiers)	Non	Non	Non	Non

Légende : Oui = répond pleinement au critère ; Partiel = y répond avec des contournements ; Non = n'y répond pas.

Décision

Nous adoptons Drizzle ORM, avec `drizzle-kit` pour les migrations, sur MySQL 8.4 comme seul moteur.

L'argument décisif est C1 : l'outbox transactionnel est le composant dont dépend la fiabilité des événements, et Drizzle est la seule option qui l'exprime entièrement avec l'API typée, tout en apportant schéma et migrations.

Dans l'audit, Prisma était surtout justifié par la possibilité de passer de SQLite à MySQL en changeant la configuration. Cet argument ne tient plus puisque SQLite est abandonné.

Conséquences

Positives

- Une seule couche d'accès aux données : suppression de `persistence/sqlite.ts`, `persistence/mysql.ts` et du pool séparé de `infrastructure/db/mysql.ts`.
- Des migrations versionnées et relues en PR, qui remplacent les `CREATE TABLE IF NOT EXISTS`.
- L'outbox et `updateTask` sont typés de bout en bout.
- L'image Docker perd `sqlite3` et sa chaîne de compilation native (python, make, g++), ce qui réduit aussi la surface d'attaque signalée par l'audit (section 3.6).
- Les schémas Zod des routes peuvent être dérivés des tables.

Négatives / risques

- **Maturité** : l'API évolue encore. → Figer les versions exactes de `drizzle-orm` et `drizzle-kit`, et faire les mises à jour dans des PR dédiées.
- **Base de production existante** : une migration générée « depuis zéro » essaierait de recréer les tables. → Créer le point de départ avec `drizzle-kit pull` et le marquer comme appliqué sans l'exécuter.
- **Montée en compétence** de l'équipe sur Drizzle.
- **Changer de moteur à l'avenir** demanderait de réécrire le schéma. C'est un risque accepté.
- **Documentation à mettre à jour** : section 7.2 de l'audit, commentaires qui citent

Prisma (`Dockerfile` , `src/infrastructure/db/mysql.ts`).

Plan de mise en œuvre

L'app étant en production, chaque étape est une livraison séparée qu'on peut annuler.

0. **Filet de sécurité.** Tests d'intégration contre un vrai MySQL 8.4 dans la CI, qui passent sur le code actuel. Sauvegarde de la base avec restauration testée.
1. **Fusion des données.** Si SQLite contient encore des données, arrêter l'app, puis lancer `npm run db:merge-sqlite` en simulation, et enfin avec `--apply` . Le script est sans perte : il ne fait que des ajouts, met les cas ambigus dans `todo_items_merge_conflicts` et vérifie tout avant de valider la transaction.
2. **Point de départ du schéma.** `drizzle-kit pull` sur une copie de la prod, puis une migration de départ marquée comme appliquée. Vérifier qu'une génération suivante produit une migration vide.
3. **Nouvelle implémentation de Persistence** avec Drizzle, derrière `PERSISTENCE_DRIVER=legacy|drizzle` (`legacy` par défaut). On bascule par variable d'environnement, sans redéployer.
4. **Outbox.** Migrer `withTransaction` , `enqueue` , `claimBatch` , `markPublished` et `updateTask` , avec la même bascule. Surveiller le vidage de `outbox_events` et la file RabbitMQ.
5. **Nettoyage**, après une période stable : supprimer les adaptateurs historiques, `sqlite3` , la variable de bascule et les créations de tables à la main. Simplifier le `Dockerfile`.
6. **Renforcement du schéma** en *expand* / *contract* : clé primaire et `NOT NULL` sur `todo_items` , après avoir traité les doublons et les valeurs `NULL`.

Règles transverses : les migrations tournent dans une étape de déploiement dédiée, jamais au démarrage de l'app. Chaque migration reste compatible avec la version précédente du code. Le SQL généré est relu en PR.

Critères de réussite

- Plus aucune requête SQL écrite à la main hors des migrations. Les seules exceptions éventuelles sont documentées.
- Les tests d'intégration MySQL passent en CI sur chaque PR.
- `drizzle-kit generate` ne produit aucune migration sur `main` (schéma et base alignés).
- L'image de production ne contient plus `sqlite3` ni sa chaîne de compilation native.
- Aucune perte de données constatée : le nombre de lignes avant et après chaque étape est vérifié.

Revoir cette décision si

- un moteur autre que MySQL redevient nécessaire ;
- Drizzle n'est plus maintenu ou introduit des changements incompatibles répétés ;
- l'outbox est déplacé vers un service qui n'utilise plus cette base.